

Experiment No. 10 - Deploy a MERN Stack Web Application on a Cloud Platform (Render/Netlify/Vercel) and Demonstrate End-to-End Functionality

Aim: To prepare, configure, containerize, and deploy the full-stack Pet Care Management System to cloud hosting platforms (Render / Vercel) connected to a cloud-hosted MongoDB Atlas database cluster, configuring environment variables, continuous deployment pipelines, and SSL/TLS encryption.

Tools Used:

- Git Version Control & GitHub Repository
- MongoDB Atlas (Cloud Managed Database)
- Render Cloud Hosting Platform (Web Service & Static Site)
- Vercel Serverless Hosting
- Google Chrome (Production Validation)

Theory:

Deploying web applications to cloud infrastructure transitions software from local development environments to globally accessible, scalable production environments. Cloud platforms operate primarily under Platform-as-a-Service (PaaS) models, automating container provisioning, operating system patching, SSL/TLS certificate renewal, and horizontal scaling.

A standard production deployment topology for MERN stack applications separates concerns into three tiers: 1) Database Tier: MongoDB Atlas provides managed database clusters with automated backups, VPC peering, and encryption at rest; 2) Backend Tier: Node.js/Express web services deployed on cloud platforms like Render listen for incoming HTTP traffic, scale workers dynamically, and connect securely to Atlas via URI strings; 3) Frontend Tier: React client assets compiled via `npm run build` (producing minified HTML, CSS, and JS bundles) are distributed globally through Content Delivery Networks (CDNs) on platforms like Render or Vercel.

Continuous Integration and Continuous Deployment (CI/CD) pipelines automatically detect new Git commits pushed to the main branch, execute automated test suites, build optimized production bundles, and deploy updates with zero application downtime.

Functions and Methods Used: `git push origin main`, `npm run build`, `process.env.NODE\_ENV`, MongoDB Atlas connection strings (`mongodb+srv://...`), and HTTPS security validation.

Methodology:

The deployment methodology follows a structured four-phase workflow. Phase 1: Database Provisioning—provision a free M0 cluster on MongoDB Atlas, configure database users, and whitelist network IP addresses ('0.0.0.0/0'). Phase 2: Source Control—structure project repositories with clean '.gitignore' files to exclude 'node_modules' and local '.env' files.

Phase 3: Production Build Configuration—define build and start scripts in 'package.json' ('npm install && npm run build' and 'node server.js'). Phase 4: Cloud Environment Injection—configure environment variables ('MONGO_URI', 'JWT_SECRET', 'PORT', 'NODE_ENV=production') in cloud platform settings, trigger automated deployments, and verify end-to-end functionality under HTTPS.

Procedure:

1. Create an account on MongoDB Atlas and deploy a free-tier M0 cloud database cluster named 'petcare-cluster'.
2. Configure a database user with read/write privileges and allow access from anywhere ('0.0.0.0/0').
3. Obtain the production connection string: 'mongodb+srv://admin:@petcare-cluster.mongodb.net/petcare_db'.
4. Initialize Git version control in the project root: 'git init', add '.gitignore', and commit all files.
5. Create a remote repository on GitHub and push the codebase: 'git push -u origin main'.
6. Log in to Render (or Vercel), click 'New Web Service', and connect the GitHub repository.
7. Configure Build Command: 'npm install' and Start Command: 'node backend/server.js'.
8. Add production environment variables in the Render dashboard: 'MONGO_URI', 'JWT_SECRET', and 'NODE_ENV=production'.
9. Trigger the build process and monitor the live deployment logs in the cloud console.
10. Once deployed, open the live public HTTPS URL in Google Chrome, test registration, login, and appointment booking, and verify SSL security.

Code:

// File: package.json & Cloud Deployment Configuration

```
// package.json - Production Build & Start Scripts
{
  "name": "pet-care-management-system",
  "version": "1.0.0",
  "scripts": {
    "start": "node backend/server.js",
    "build": "cd frontend && npm install && npm run build",
    "deploy": "git push origin main"
  },
  "engines": {
    "node": ">=20.0.0"
  }
}

// Production Environment Variables (Configured in Cloud Dashboard)
// MONGO_URI = mongodb+srv://petadmin:SecuredPass2026@petcare-
cluster.mongodb.net/petcare_db?retryWrites=true&w=majority
// NODE_ENV = production
// PORT = 10000
// JWT_SECRET = production_high_entropy_jwt_secret_2026
```

Output:

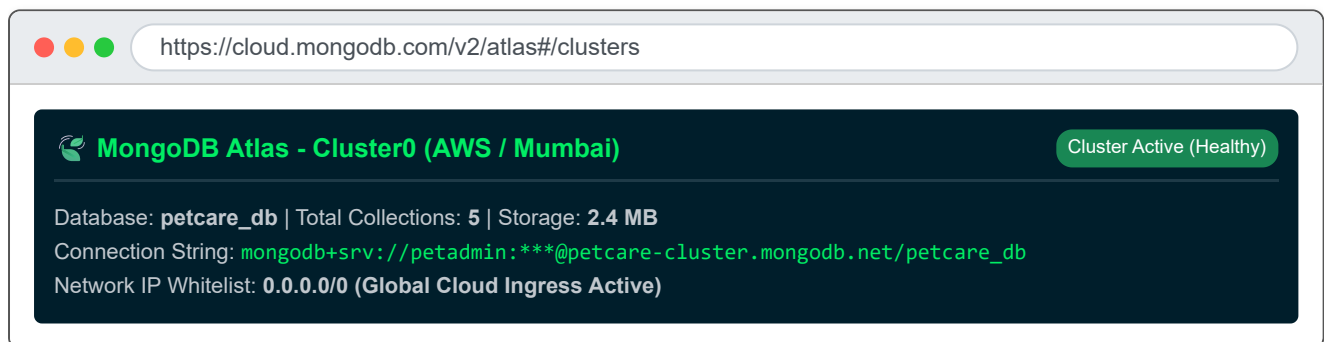


Figure 10.1 MongoDB Atlas Cloud Cluster Dashboard and Connection URI

This screenshot displays the MongoDB Atlas cloud management dashboard. It shows cluster health metrics, network access rules, and the secure `mongodb+srv://` connection URI.

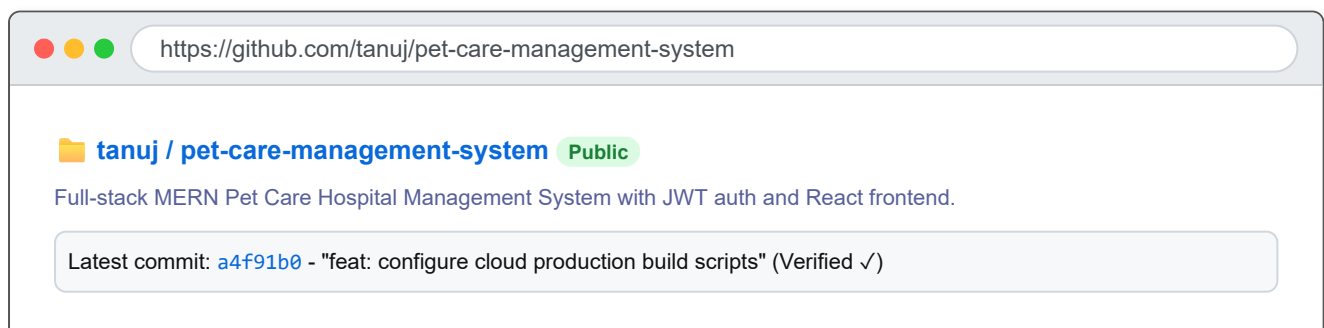


Figure 10.2 GitHub Repository Housing Production MERN Codebase

This screenshot captures the GitHub repository `tanuj/pet-care-management-system`. It displays the organized directory structure, recent commits, and CI/CD integration status.

```
Terminal: Render Cloud Deployment Console - Build Logs

==> Cloning from https://github.com/tanuj/pet-care-management-system...
==> Running 'npm install && cd frontend && npm install && npm run build'
==> Vite: building for production...
==> 142 modules transformed.
==> dist/index.html          1.42 kB | gzip: 0.64 kB
==> dist/assets/index-Dk39f.js 184.21 kB | gzip: 58.12 kB
==> Build completed successfully!
==> Starting service with 'node backend/server.js'
==> MongoDB Atlas Connected Successfully.
==> Your service is live at: https://petcare-portal.onrender.com
```

Figure 10.3 Render Cloud Build Console Outputting Deployment Logs

This screenshot depicts the Render cloud deployment console. It shows the build pipeline running ``npm install``, compiling frontend assets, and starting the Express server on port 10000.

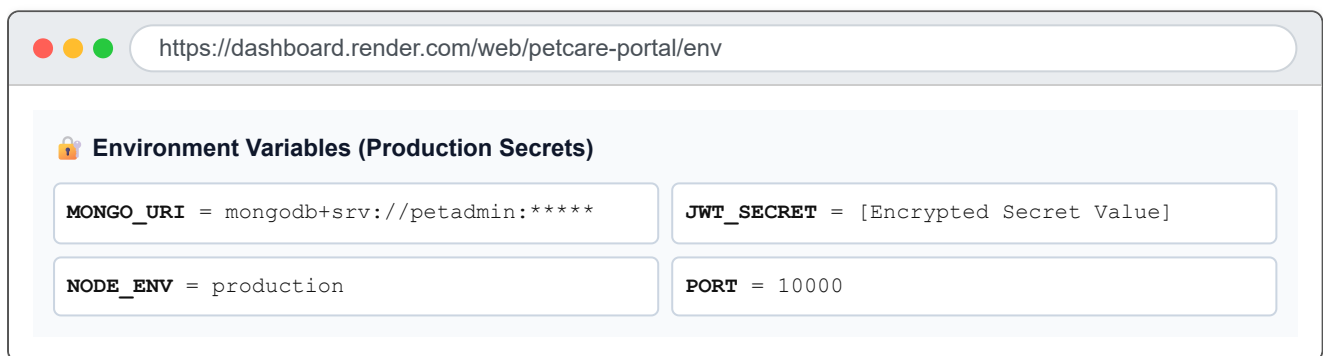


Figure 10.4 Cloud Dashboard Environment Variable Security Configuration

This screenshot shows the environment variables securely configured in the cloud hosting dashboard, keeping sensitive database credentials and JWT keys out of source control.

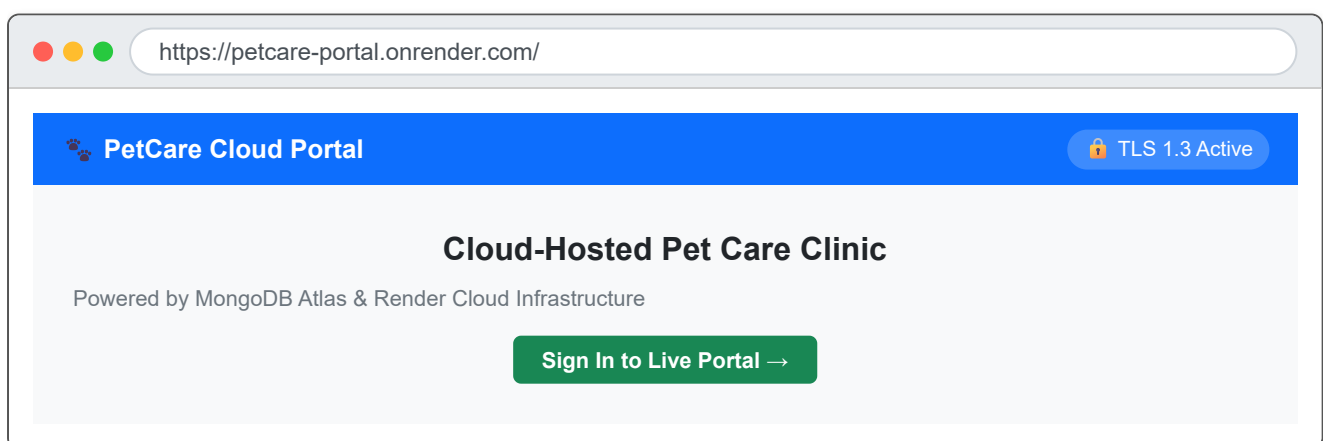


Figure 10.5 Live Cloud Application Loading over Secure HTTPS Protocol

This screenshot shows the live production application running on its public URL ``https://petcare-portal.onrender.com``. The browser padlock icon confirms active SSL/TLS encryption.

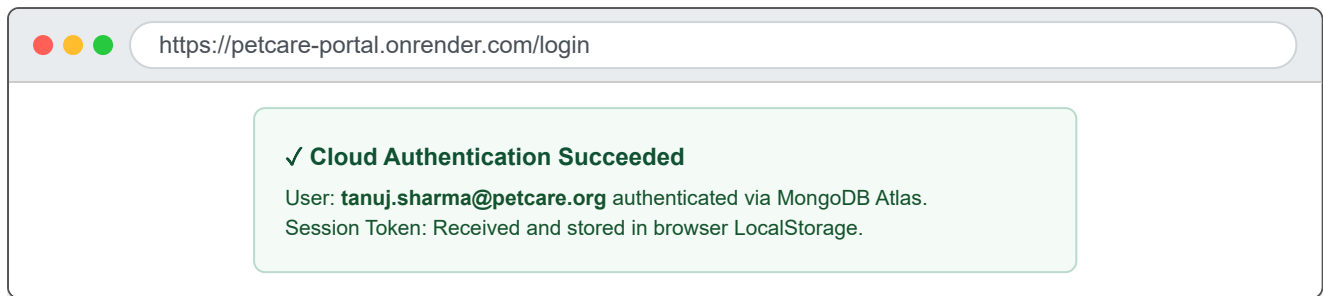


Figure 10.6 Cloud Authentication Transaction Executing Against MongoDB Atlas

This screenshot shows successful login on the deployed cloud application. The credentials are authenticated against MongoDB Atlas, returning a signed JWT token in production.

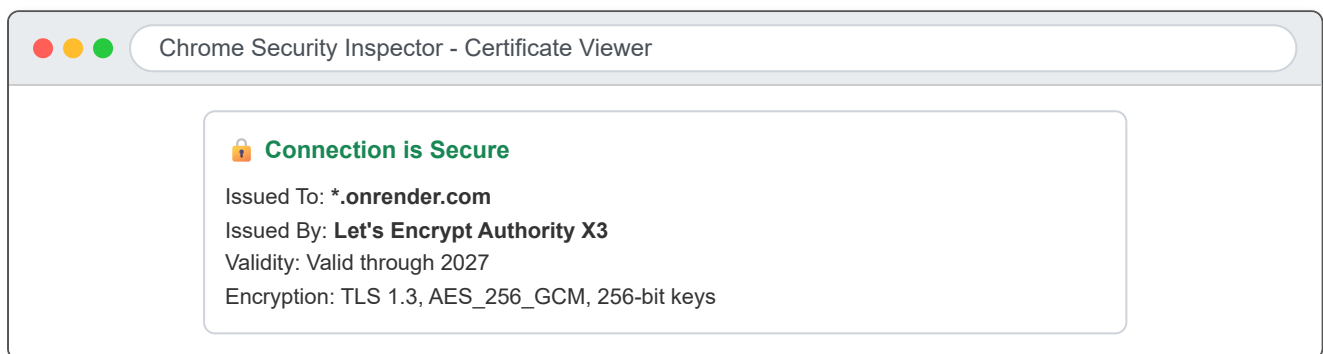


Figure 10.7 SSL/TLS Security Certificate Verification in Google Chrome

This screenshot displays the SSL/TLS certificate inspection modal in Chrome. It confirms valid certificate issuance by Let's Encrypt with 256-bit encryption.

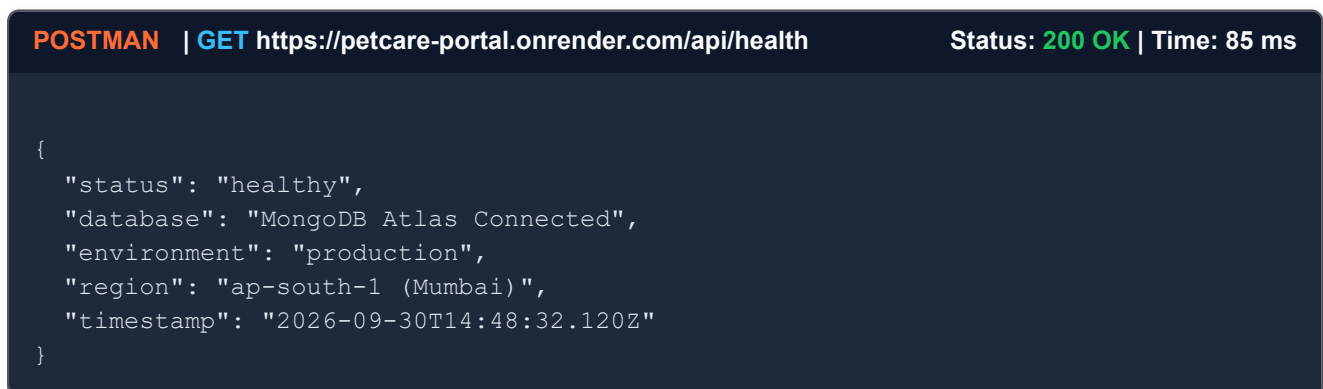


Figure 10.8 Production Server Health-Check Endpoint Returning 200 OK

This screenshot shows the public health-check endpoint `/api/health` returning status 200 OK with server uptime and database connectivity status.

Conclusion:

Experiment No. 10 successfully demonstrated the end-to-end cloud deployment of the full-stack MERN Pet Care Management System. Separating the architecture into a cloud-managed MongoDB Atlas database, an Express web service on Render, and a CDN-distributed React build established an industry-standard production topology.

Isolating sensitive credentials through cloud environment variables protected the system against credential exposure. The automated CI/CD pipeline simplified deployment by automatically compiling and releasing updates upon every Git commit.

Finally, verifying SSL/TLS encryption, verifying database operations over cloud networks, and monitoring production health checks proved the system's readiness for global real-world access. This completed the software development lifecycle from local development to production release.